

Topic 6: Operator Overloading

6.1 What is Operator Overloading?

- C++ lets you redefine how operators (+, -, *, ==, <<, etc.) work for user-defined classes
- Makes class objects behave like built-in types: `v1 + v2` instead of `v1.add(v2)`
- Implemented as member functions or global (friend) functions
- You cannot change operator precedence or create new operators
- Cannot overload: `::` (scope), `.` (dot), `.*` (pointer to member), `?:` (ternary), `sizeof`

6.2 Overloading Arithmetic Operators

- Return a new object (by value) -- do not modify the operands
- As member function: left operand is the calling object (`*this`), right operand is the parameter
- As global friend: both operands are parameters -- needed when left operand is not your class

```
class Vector2D {
    double x, y;
public:
    Vector2D(double x=0, double y=0) : x(x), y(y) {}

    Vector2D operator+(const Vector2D& other) const {
        return Vector2D(x + other.x, y + other.y);
    }
    Vector2D operator*(double scalar) const {
        return Vector2D(x * scalar, y * scalar);
    }
    void print() const { cout << "(" << x << ", " << y << ")"; }
};
```

6.3 Overloading Comparison Operators

- Return `bool` -- allows objects to be compared with `==`, `!=`, `<`, `>`, etc.
- Sort algorithms and standard containers rely on `<` being defined correctly

```
bool operator==(const Vector2D& o) const {
    return x == o.x && y == o.y;
}
bool operator<(const Vector2D& o) const {
    return (x*x + y*y) < (o.x*o.x + o.y*o.y); // compare magnitudes
}
```

6.4 Overloading Stream Operators

- Overload << to print objects with cout, and >> to read objects with cin
- Must be global friend functions because left operand is ostream/istream, not your class
- Return the stream by reference so that chaining works: cout << a << b;

```
friend ostream& operator<<(ostream& os, const Vector2D& v) {  
    os << "(" << v.x << ", " << v.y << ")";  
    return os;  
}  
  
// Usage: cout << v1 << " and " << v2 << endl;
```

6.5 Overloading the Assignment Operator

- The default assignment (=) does a shallow copy -- problematic when the class owns heap memory
 - Must handle self-assignment (if (this == &other) return *this;)
 - Free old resources, allocate new ones, copy data, return *this
-